What is linked list data structure ?

----

* Linked list is a linear data structure which is **internally collection of nodes.**

* Each node **is interlinked with another node in a non - contiguous memory location.**

* This linkage facility among the nodes provides an efficient way to insert OR delete an element any where in the list.

Types of linked list :

----

* There are 3 types of linked list

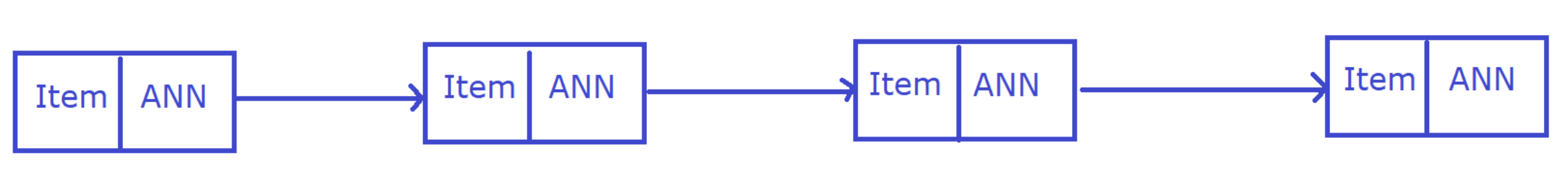
  a) Singly linked list [In Forward direction only]

  b) Doubly linked list [Forward & Backward both the direction]

  c) Circular linked list

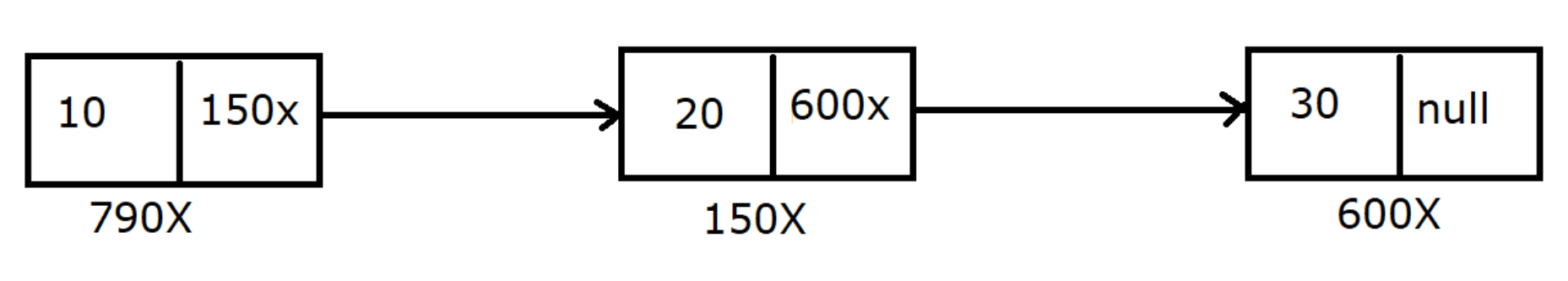
ANN = Address of next node
APN = Addtess of previous node

Singly linked list :

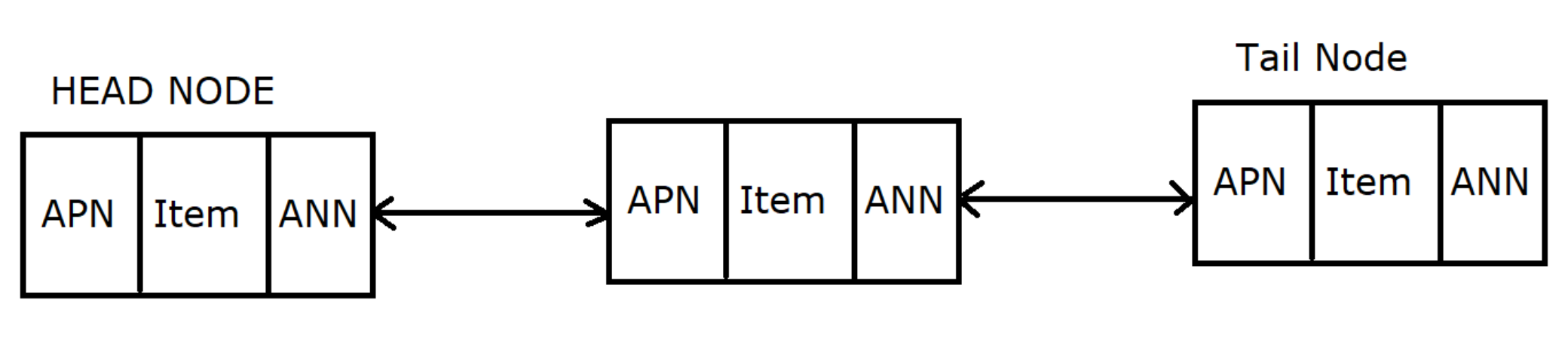


Coding implementation of singly linked list by using inner class Node :

```java
 package com.ravi.arraylist;

 public class CustomSinglyLinkedList
 {
     private static class Node
     {
         int item;
         Node next;                          //next -> Address of next node

         public Node(int item)
         {
             this.item = item;
             this.next = null;
         }
     }

     public static void main(String[] args)
     {
         Node n1 = new Node(10);
         Node n2 = new Node(20);
         Node n3 = new Node(30);

         //Establishing the link
         n1.next = n2;
         n2.next = n3;

         IO.println(n1.item);
         IO.println(n1.next.item);
         IO.println(n1.next.next.item);
      }

   }
```



Doubly linked list :



* In java, there is a predefined class called LinkedList available in java.util package from JDK 1.2V.
* It uses doubly linked list data structure

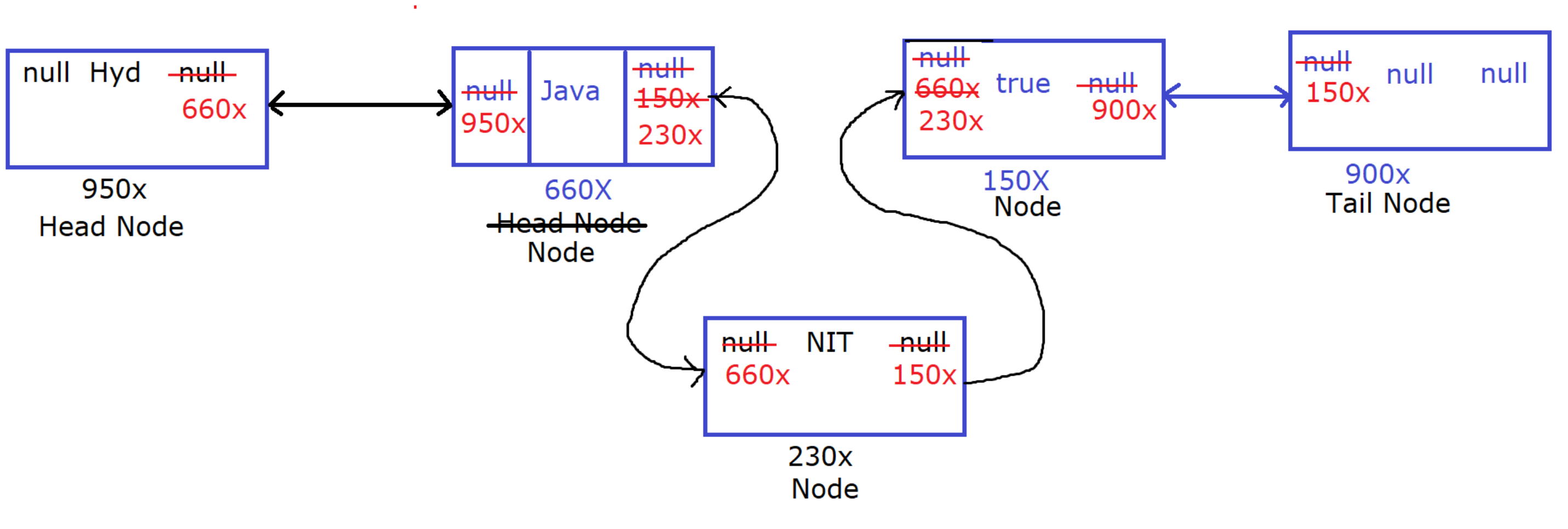
Internal working mechanism of java LinkedList class :

```
public class LinkedList<E>
{
  E item;
  Node previous;  [APN]
  Node next;      [ANN]

  public boolean add(Element e)
  {
     Node node = new Node(previous, item , next);
  }
}
```

Whenever we will call add method internally one Node object will be created to support doubly linked list

Diagram :

```
LinkedList<Object> list = new LinkedList<>();
list.add("Java");
list.add(true);
list.add(null);
list.add(1, "NIT");
list.addFirst("Hyd");
```



Method Searching Criteria :

Method	Searching Critera
indexOf()	Head Node
lastIndexOf()	Tail Node
contains()	Head Node
get()	Head OR Tail Node (Which index is closer)

LinkedList<E> :

----

public class LinkedList<E> extends AbstractSequentialList<E> implements List<E>, Cloneable, Serializable, Deque<E>

* Implemented class of List<E> interface in java.util package from JDK 1.2V
* Can accept duplicate, null,homogeneous and hetreogeneous objects.
* It stores the elements based on the **index position but in a non contiguous memory** location.
* It uses "Doubly linked list" data structure with default capacity is 0.
* It uses node reference where each node is interlinked with another node.
* This interlinked of node provides an easy way to insert OR delete an element in the list.
* Whenever we call add() method, internally Node object is created.
* Methods are not synchronized.
* Iterator is Fail Fast Iterator.
* It **may iterate the Objects more slowly than ArrayList** but it is highly recomended for insertion & deletion.
* It provides O(1) time complexity for insertion OR deletion but O(n) time complexity for retrieval operation.
* It implements from Deque<E> to support basic queue opetaion.
* It does not implement RandomAccess interface.
* At the time of searching an element, It will start searching from Head node OR tail node means closer one based on the index.

Constructor:

It has 2 constructors

1) LinkedList list1 = new LinkedList();
   It will create a LinkedList object with 0 capacity.

2) LinkedList list2 = new LinkedList(Collection c);
   Interconversion between the collection

Methods of LinkedList class:

----

1) void addFirst(Object o)
2) void addLast(Object o)

3) Object getFirst()
4) Object getLast()

5) Object removeFirst()
6) Object removeLast()

******What is difference between ArrayList & LinkedList :**

Array List	Linked List
Dynamic Array is used to implement ArrayList internally in JAVA.	Doubly Linked List is used to implement LinkedList internally in JAVA.
Array List acts only as a list because it implements a list interface.	LinkedList acts as a list as well as a queue because it implements both interfaces.
Preferred for storing and accessing the data.	Preferred for manipulation of data.
Array List provides faster random access time due to direct array indexing.	LinkedList is slower for random access but efficient for insertions and deletion in the middle.
Array List is less efficient for the insertions and deletions in the middle because elements need to be shifted.	LinkedList excels in insertions and deletions at any position due to its structure.
Storage of elements is performed at a contiguous location.	Storage of elements is performed at a non-contiguous location.
Being dynamic in nature, Array List is resizable.	LinkedList is implemented by pointing to elements thereby no sizing issues are faced.
The default capacity upon initializing is set as 10 with resizing done upon extension of the list.	The is no default capacity as insertion is performed by interlinking of the nodes and no contiguous allocation of space.
Insertion and deletion operations are slow in ArrayList as rearrangement is required.	LinkedList is much faster in terms of insertion and deletion operations.
Array List is faster when iterating sequentially through elements.	While iterating through elements LinkedList is slower due to the need to follow node references.